From Scratch:
The Design and Implementation of a SAT Solver From Scratch

Oliver Lie

July 30, 2026

Contents

Preface

Hello! Thank you for taking me seriously enough to pick up this book and begin reading it.

Before we continue, I just want you to know that I am not an expert when it comes to things such as algorithms, data structures, general computer science, or even SAT solvers (what this book is about). Despite that, that's exactly why I wanted to write this. These things that I am not an expert in are my passion, and by doing this technical research, I hope to become a little less of "not an expert" in those fields.

The aim of this book isn't to improve or revolutionize SAT solving algorithms. Rather, it is to make the methods and reasoning more accessible to the lay undergraduate (I am open to the possibility that I myself may be the lay undergraduate, and you are simply smarter than me).

Throughout this book we will build our own modern SAT solver from scratch, complete with all the bells and whistles. We will begin with the most basic algorithm imaginable—brute force—and gradually work our way through DPLL, CDCL, and beyond.

Throughout this journey we will encounter thought experiments, exercises, and a handful of experiments intended to solidify our understanding. Any external resources will be cited (hopefully correctly), and in keeping with the goal of accessibility:

- Questions that are explicitly asked will eventually be answered.
- Proofs will be explained in informal language before becoming formal.
- Code will be provided and linked through GitHub (or another hosting platform).
- Revisions to this book will be made whenever I discover mistakes or better explanations.

There is one more thing that deserves acknowledgement: some pieces of code were AI-assisted.

To be completely transparent, "AI-assisted" means that instead of spending days repeatedly banging my head against a wall trying to understand a bug or some subtle algorithmic detail, I occasionally asked an AI to explain concepts or help debug an implementation. Every word of this book, however, is my own. AI did not generate any of the explanatory text you will read here, although it certainly helped produce some of the code that accompanies it.

There are practical reasons for this. SAT solvers are difficult to implement correctly, and introducing subtle bugs is remarkably easy. In fact, before deciding to write this book, I had already attempted to build a SAT solver once before. I never managed to get a working CDCL implementation.

That failure, however, taught me a tremendous amount.

So in a sense, we are learning SAT solving together (isn't that comforting?). My previous mistakes have shown me many of the pitfalls, and my hope is that by documenting this journey I can help you avoid making the same ones.

So without further ado, let's begin.

By the time we reach the end, I hope I will have taught you more than I myself learned.

Chapter 1

Introduction

SAT solvers, in my opinion, are one of the most underrated pieces of algorithmic technology of the modern day. They are used in both software and hardware verification (circuitry testing), product configuration, package management, computational biology, cryptanalysis, particle physics, solving graph problems, and much much more.[1]

In a more computer science-focused world, they have applications in NP-Completeness. Since (nearly) every problem has a reduction to a Boolean SAT problem, creating one really really good SAT solver lets us solve all those problems in a "fast" manner. So they are important everywhere despite the vast majority of us never really thinking about their existence.

One note on what "fast" means: even if solving a SAT problem takes a small amount of time, the algorithmic complexity isn't necessarily "fast" as there is no known polynomial algorithm for any NP-Complete problem (I am a firm believer that $P = NP$, and yes I know that makes me crazy).

Before we start writing out code, proofs, and the like, we need to make sure we're all on the same page on things such as input and ideas. Reading through this introduction is important so that you understand the basic notation used throughout this book.

One last thing to note: we will write out pseudocode rather than the code for one specific programming language. However, although the idea behind pseudocode is that it's programming language agnostic (it doesn't carry artifacts of any real programming language (usually)), this idea is poorly executed when it comes to formatting.

My algorithms professor will probably scream at me for this, but I'm making the executive decision that we will follow a general C-based syntax for readability purposes. That is, we will use braces {} for block scope, parentheses () for grouping, `=` for assignment, `==` for equality, and `<=`, `>=`, etc. for their respective uses. Everything that can be inferred visually will be used.

Another thing to note is that we will pretend our arrays can dynamically resize, so we won't pre-allocate space, and we will also assume 1-based indexing for an easier map from variables to indices, as you will see later.

1.1 What Do SAT Solvers Even Solve?

"What do SAT solvers even solve," I hear you say? SAT solvers solve Boolean satisfiability problems. Another way to say that is they solve propositional logic problems. There are many forms of propositional logic, and you may be familiar with it. Here are some basic examples:

- $P \to Q$, an implication statement
- $(A \wedge B)$, an and statement
- $(A \vee B)$, an or statement
- $\neg A$, a negation/not statement

[1] `https://www.cs.utexas.edu/~isil/cs389L/lecture4-6up.pdf`

Of course these are very basic examples, and we haven't included every operator such as xor, nand, etc. As you know, we can rewrite implications to contain only and, or, and not operators, which is what SAT solvers operate on. In other words, SAT solvers only operate on Boolean algebra equations.[2]

However, SAT solvers don't operate on just any Boolean algebra equation.

No no no, they must be in a proper form, called CNF (more on that in the next section). CNF (Conjunctive Normal Form) equations are a "conjunction of disjunctions," i.e., clauses of ors, combined with ands.

Some examples of CNF are as follows:

- $(A \vee B)$
- $(A \vee B) \wedge (\neg C)$
- $(A \vee B) \wedge (\neg C \vee D)$

Some examples that are **not** CNF:

- $\neg(A \vee B)$, where there is a NOT in front of a clause
- $\neg(A \vee B) \wedge (C)$, where there is a NOT in front of a clause
- $(A \vee B \wedge C)$, where there is an AND within a clause
- $(A \wedge B) \vee (C)$, where it is in DNF form.[3]

1.2 CNF Encoding

Now that we know what our inputs look like, the next question (I hope you're asking questions) is "how do we encode that?" Well good thing you're reading this, because that's what we're here to talk about! In this section, we'll expand our base of boolean algebra, including some definitions, transformations, and finally good encoding.

Definitions We'll Use

Clause A clause is a collection of variables, in which each variable is separated by an or ($\vee$), and each clause is separated by an and ($\wedge$).

Variable A variable is an element of a clause, taking on a value of True, False, or Unassigned. Each variable can be negated ($\neg$) or not, affecting its evaluated value.

Literal A literal is an evaluated variable, evaluating to one of True, False, or Unassigned. Since a literal is an evaluated variable, it cannot be negated. A literal whose corresponding variable is unassigned evaluates to Unassigned, regardless of whether or not the variable is negated.

A quick example of the difference of a literal value, using all three of our definitions:

$$(\neg A)$$

is a clause of one variable "A", which is negated. If the variable A = False, then its literal value is True, because

$$\neg\text{False} \equiv \text{True}.$$

In other words, A was assigned the value False, and then it evaluated to True. This will be very important in the future, so please take the time to understand it well enough.

[2] A Boolean algebra equation is one containing only and ($\wedge$), or ($\vee$), and not ($\neg$) operations (at a basic level).

[3] DNF (Disjunctive Normal Form) is a "disjunction of conjunctions," i.e., clauses of ands combined by ors. Solving these is trivially easy, as we just need to find one clause where no element evaluates to false.

As discussed earlier, a CNF equation only has three operators: and ($\wedge$), or ($\vee$), and not ($\neg$), but in propositional logic, there are many more operators, and their clauses are not guaranteed to be in the format we want. There are many algorithms to transform any propositional logic equation into CNF, but frankly, that's beyond the scope of this book (it's also beyond my scope). Instead, we will just look at how we transform each operator into an equivalent format using just our three operators.

Implication ($\rightarrow$)

p	q	$\neg p$	$p \rightarrow q$	$\neg p \vee q$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Xor ($\oplus$)

p	q	$\neg p$	$\neg q$	$p \oplus q$	$p \vee q$	$\neg p \vee \neg q$	$(p \vee q) \wedge (\neg p \vee \neg q)$
T	T	F	F	F	T	F	F
T	F	F	T	T	T	T	T
F	T	T	F	T	T	T	T
F	F	T	T	F	F	T	F

Biconditional ($\leftrightarrow$)

p	q	$p \leftrightarrow q$	$p \rightarrow q$	$q \rightarrow p$	$(\neg p \vee q) \wedge (\neg q \vee p)$
T	T	T	T	T	T
T	F	F	F	T	F
F	T	F	T	F	F
F	F	T	T	T	T

We won't look at negations of operators such as nand, nor, xnor, or any others. I'm not a boolean algebra expert when it comes to what operators exist. But if you were given an equation with these extra operators, you could easily apply the correct equivalency, then apply an algorithm that converts any boolean algebra equation into one of CNF form.[4]

DIMACS CNF Format

Now that we know what CNF looks like, we can now discuss how it's formatted for a SAT solver's consumption. Typically, CNF is formatted in *DIMACS CNF* form.

Some rules for DIMACS CNF form are:

1. The file may begin with comment lines. The first character of each comment line must be a lower case letter "c". Comment lines typically occur in one section at the beginning of the file, but are allowed to appear throughout the file.

2. The comment lines are followed by the "problem" line. This begins with a lower case "p" followed by a space, followed by the problem type, which for CNF files is "cnf", followed by the number of variables followed by the number of clauses.

3. The remainder of the file contains lines defining the clauses, one by one.

4. A clause is defined by listing the index of each positive literal, and the negative index of each negative literal. Indices are 1-based, and for obvious reasons the index 0 is not allowed.

5. The definition of a clause may extend beyond a single line of text.

[4] I honestly may be talking out of my ass here. I will do more research on this subject at a later date.

6. The definition of a clause is terminated by a final value of "0".
7. The file terminates after the last clause is defined.

There are some things about this syntax to be wary of:

1. The definition of the next clause normally begins on a new line, but may follow, on the same line, the "0" that marks the end of the previous clause.
2. In some examples of CNF files, the definition of the last clause is not terminated by a final "0".
3. In some examples of CNF files, the rule that the variables are numbered from 1 to N is not followed. The file might declare that there are 10 variables, for instance, but allow them to be numbered 2 through 11.
4. Comments can appear anywhere in the file, but the comment must be on its own line and it must start with "c".[5]

For a quick example, here is a simple way to format a DIMACS CNF equation:

```
p cnf 4 2
1 2 -3 0
-1 4 0
```

And one with comments:

```
c this is a comment
p cnf 4 2
c this is another comment
1 2 -3 0
-1 4 0
```

In order to increase the compatibility of our SAT solver, we will be bending some rules.

We will firstly allow missing header lines. In the case where no header is passed in, we will instead infer the number of clauses and variables. We will also be very lenient in where comments can appear, but we will enforce that variables must be between 1 and N.

1. Comments can appear anywhere in the file, but they **MUST** be on their own line and start with "c".
2. A header is not necessary, and if one is not provided, then the number of variables and clauses shall be inferred. However, if one is provided, then the number of variables and clauses **MUST** match the header.
3. All clauses, with the exception of the last clause, must end with "0", and can extend multiple lines.
4. All variables are numbered 1 through N, where N is the number of variables. That is, if you say there are 10 variables, they must be labeled $1, 2, \ldots, 10$ and not $2, 3, \ldots, 11$ or any other convention.
5. (For compatibility) Upon encountering a "%" character outside of a comment, we will stop parsing immediately.

Thus, valid input can take on many forms:

```
p cnf 4 2
1 2 -3 0
-1 4 0

4 2
1 2 -3 0
-1 4 0
```

[5] https://jix.github.io/varisat/manual/0.2.0/formats/dimacs.html

```
p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4 0


p cnf 4 2
c this comment doesn't have to be at the top
1 2 -3 0
-1 4
c the first two numbers are still the number
c of variables and the number of clauses


p cnf 4 2 1 2 -3 0
-1 4 0
```

More information can be found here.[6]

1.3 Reading a CNF Equation Programmatically

Now that we know what our input looks like, we can finally discuss reading it programmatically. Firstly we must discuss how our SAT solver will be represented in code. At the moment, we just need to keep track of

- the number of variables
- the number of clauses
- each clause in our equation
- whether or not we've tried to solve the equation yet
- what our solution is

We also want some methods to parse the equation, construct a SAT solver, solve the equation, print out the solution if it is satisfiable, and whether or not there's a solution. Therefore, our object will look like so:

```
public interface:
    SATSolver(string input)
    bool solveCNF()

private interface:
    void parse(string equation)
    int numVars
    int numClauses
    int[][] clauses
    optional<bool>[] vars
```

Our pseudocode is as follows:

```
SATSolver(string input)
{
    vars = no-value

    if (input is file)
    {
```

[6] https://people.sc.fsu.edu/~jburkardt/data/cnf/cnf.html

```
        string content = readFile(input)
        parse(content)
    }
    else
    {
        parse(input)
    }
}

parse(equation)
{
    // The method of parsing if there's a header
    if (contains(equation, "p cnf"))
    {
        int[1..N] Arr

        for each (line in input)
        {
            if (startsWith(line, 'c'))
            {
                skip to next line
            }
            else
            {
                for each (token in line)
                {
                    append(Arr, token)
                }
            }
        }

        numVars = Arr[1]
        numClauses = Arr[2]

        idx = 2

        for (i = 1 up to numClauses)
        {
            int[1..N] clause

            while (Arr[idx] != 0)
            {
                append(clause, Arr[idx])
                idx = idx + 1
            }

            append(clauses, clause)
        }
    }

    // Method of parsing when there's no header
    else
    {
        numVars = 0
```

```
        numClauses = 0

        int[1..N] Arr

        for each (line in input)
        {
            if (startsWith(line, 'c'))
            {
                skip to next line
            }
            else
            {
                for each (token in line)
                {
                    numVars = max(numVars, token)

                    if (token == 0)
                    {
                        numClauses = numClauses + 1
                    }

                    append(Arr, token)
                }

                idx = 0

                for (i = 1 up to numClauses)
                {
                    int[1..N] clause

                    while (Arr[idx] != 0)
                    {
                        append(clause, token)
                        idx = idx + 1
                    }

                    append(clauses, clause)
                }
            }
        }
    }
}
```

This code will be implemented slightly differently in the example code, as I am using C++. There is also a big bug, which is that the code assumes that because "p cnf" appears in the file, there is a header. This is not always true because it does not consider whether or not the header is on the same line as a comment, so this will break it:

```
c p cnf
1 2 -3 0
-1 4 0
```

Because it assumes that there is a header when in fact there is not. I will leave fixing this bug as an exercise to both the reader and the writer, however, I will not be implementing this fix because if you're purposely trying to break the constructor, you're not trying to solve a CNF equation anyways. The fix, in case you're wondering, would be to just scan the lines and for each line that doesn't start with "c," check

that it starts with "p cnf," and if it does, then you know for certain that there is a header. Now the real exercise is to determine whether or not there's a header in just one pass instead of two.

Chapter 2

Brute Force

Since the dawn of man, whenever encountered with a difficult problem, we have decided "eh, let's do it in the hardest way possible because I can't think of a better solution." The history of brute forcing things goes all the way back to our ancestors discovering fire,[1] all the way to our current solution to climate change.[2]

The idea of brute forcing something is really as simple as it gets. We have n number of literals, and therefore 2^n possible assignments, and we will check them all until we have either found a solution, or until we've run out of assignments to check. Because of how simple the idea is, it would just be better to show code and explain that instead of explaining then showing code.

```
solveCNF()
{
    bool[1..numVars] Assignment

    for (i = 0 up to 1 << numVars)
    {
        for (j = 0 up to numVars - 1)
        {
            Assignment[j] = i & (1 << j)
        }

        bool solved = true

        for each (clause in clauses)
        {
            bool satisfied_clause = false

            for (j = 1 up to clause.size)
            {
                int variable = clause[j]
                int v_idx = abs(variable)

                if (variable > 0)
                {
                    satisfied_clause =
                        Assignment[v_idx] || satisfied_clause
                }
                else
                {
                    satisfied_clause =
```

[1] This is probably not true.

[2] It's doing nothing and hoping everything sorts itself out, which is really disappointing.

```
                        !Assignment[v_idx] || satisfied_clause
                }
            }

            solved &= satisfied_clause
        }

        if (solved == true)
        {
            vars = Assignment
            return true
        }
    }

    return false
}
```

Unfortunately, the formatting across the page kind of sucks, but let's try to explain it anyways.

First, we create an array to hold each boolean value, and then next we start our loop from 0 all the way to "1 « numVars." If you are unfamiliar with what the left bit shift operator ('«') is, essentially, it is a fast way of computing some number times 2 times the number of places to shift. That is all the explanation I am going to give because that's not the point. But by computing '1 « numVars', we are computing $1 \rightarrow 2^n$, which happens to be the total number of possible variable combinations (from all False to all True).

Then in the first inner for loop, we put the current variable combination into the 'Assignment' array. The index variable 'i' holds the current variable combination, and we isolate the jth variable using '1 « j' and put it into the jth spot in the 'Assignment' array.

Next, we evaluate the CNF equation against the current assignment. We first assume that the equation is solved (innocent until proven guilty), and then we evaluate each clause. We initially assume the clause is false because logically, if one literal is True, then OR-ing it with False makes the "clause satisfied" condition true (we could exit out of the evaluation loop, but it's easier to read if we don't). Then, we "and" the "clause satisfied" evaluation with the "equation solved" condition.

At the end of the evaluation loop, we check if we found a satisfiable assignment. If we have, we then make our 'vars' field, which holds the solution if there is one, the assignment, and then we return True to indicate that we have found a satisfiable assignment. If each possible assignment is not satisfiable, then we return False, and 'vars' will hold nothing.

Since it's trivially easy to see why the code works, we will move onto something more interesting: optimizing brute force.

2.1 Optimizing Brute Force

I'll be completely honest with you, if you want to skip this section, go ahead. It's more or less just something interesting enough we can do to make the (arguably) worst algorithm[3] better.

Since we're dealing with pseudocode, we're not going to consider the hardware side of computers such as branch prediction, cache locality, or anything that the hardware uses to execute our code (other than the CPU). We will consider it from a pure Big-O perspective (excluding constant factors unless we're choosing between two algorithms with the same computational complexity).

The first thing we will add is short circuiting to our clause evaluation. After line 26 where we finish OR-ing the "clause satisfied" boolean, we will add in a new block:

```
...
bool satisfied_clause = false;

for (j = 1 up to clause.size)
```

[3] I do not know of any algorithm worse than brute forcing.

```
{
    int variable = clause[j]
    int v_idx = abs(variable)

    if (variable > 0)
    {
        satisfied_clause =
            Assignment[v_idx] || satisfied_clause
    }
    else
    {
        satisfied_clause =
            !Assignment[v_idx] || satisfied_clause
    }

    if (satisfied_clause == True)
    {
        continue
    }
}

...
```

This just says "once this clause is satisfied, go evaluate the next one immediately."

We can then add another conditional after this loop, which will move onto the next variable assignment the instant the equation is no longer satisfied:

```
...
bool solved = true;

for each (clause in clauses)
{
    bool satisfied_clause = false;

    for (j = 1 up to clause.size)
    {
        ...
    }

    if (satisfied_clause == True)
    {
        continue
    }

    solved &= satisfied_clause;

    if (solved == False)
    {
        break
    }
}

...
```

The next algorithm optimization we can make is removing the loop where we move the current assignment into the 'Assignment' array. Rather than doing it upon every iteration, why don't we do it once we find a

satisfiable assignment?

```
solveCNF()
{
    bool[1..numVars] Assignment

    for (i = 0 up to 1 << numVars)
    {
        bool solved = true

        for each (clause in clauses)
        {
            bool satisfied_clause = false

            for (j = 1 up to clause.size)
            {
                int variable = clause[j]
                int v_idx = abs(variable)

                if (variable > 0)
                {
                    satisfied_clause =
                        (i & (1 << j)) || satisfied_clause
                }
                else
                {
                    satisfied_clause =
                        !(i & (1 << j)) || satisfied_clause
                }

                if (satisfied_clause == True)
                {
                    continue
                }
            }

            solved &= satisfied_clause

            if (solved == False)
            {
                break
            }
        }

        if (solved == True)
        {
            for (j = 0 up to numVars - 1)
            {
                Assignment[j] = i & (1 << j)
            }

            vars = Assignment
            return true
        }
    }
```

```
    return false
}
```

With that, that's all the optimization I can think of. In case you're wondering, no this does not change the asymptotic runtime of the method, however, it's easy to see why these changes would presumably make the algorithm run faster as we're adding code that stops evaluation the moment we've figured just enough out to know if it's worth continuing evaluation and deferring extra calculation (recording the current assignment) until once we know it's satisfiable.

As an exercise to the reader, I will ask you, how can you parallelize this algorithm? And once you've figured that out, can you program it in a way where once one thread finds a satisfactory assignment, it terminates all other threads immediately instead of waiting for the others to finish?

2.2 Brute Force – Results

To see how well our current algorithm does, running some tests would be a good idea. Why run tests when we have already proven that our algorithm works "perfectly?" To see how quickly it runs of course.

To test its speed (and as we add more features, to ensure correctness is kept), we use a database of problems from here:

`https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html`

We gave our current implementation 1000 SAT instances (uf20-91), each with 20 variables and 91 clauses each, all satisfiable. It took this baseline brute force algorithm 2,119,301 milliseconds, which is roughly 35.32 minutes.

Normally in science you have to run an experiment at least three times for data accuracy. But you must understand that I don't want to give up an hour and a half of my life to an algorithm that we're going to be improving upon anyways.

Chapter 3

DPLL

The Davis–Putnam–Logemann–Loveland (DPLL) algorithm is a complete backtracking-based search algorithm for finding satisfiable assignments for SAT problems. It is, dare I say, the foundational algorithm for modern SAT solvers, as the state-of-the-art algorithm used by basically every SAT solver builds upon the foundations that DPLL provides, as we will see later on.

It consists of three components:

- Recursive backtracking
- Unit propagation
- Pure literal elimination

How does it work?

Imagine a binary tree of variables, each with two branches representing their assignment, one for true and one for false. Suppose we don't know anything about our CNF equation. That is, from looking at it, any variable could have any value, and we don't know how each assignment helps us in finding out whether or not the equation is satisfiable. How can we figure out if we can satisfy the equation without just brute forcing it?

The answer is making an ass out of you and me to make assumptions.

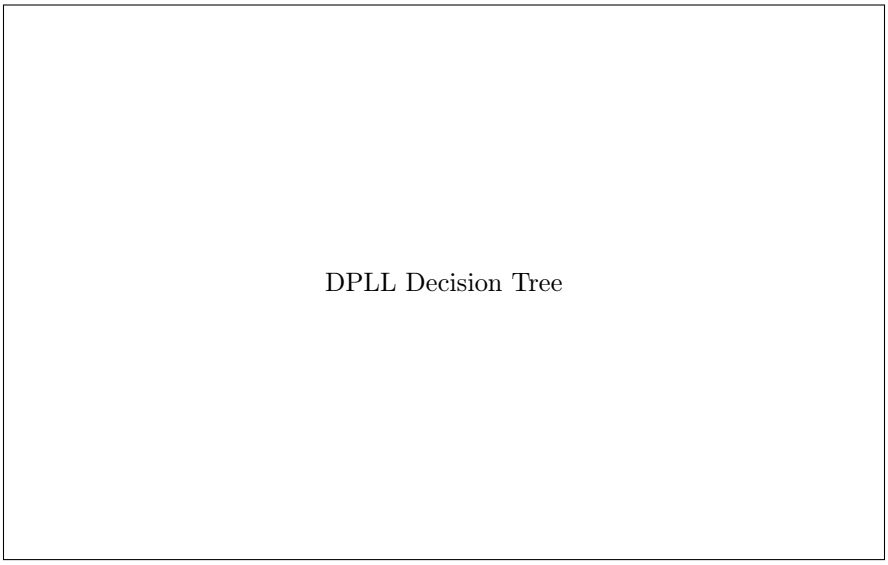

Figure 3.1: Example DPLL decision tree (the outcomes may not correspond to an actual CNF instance, sue me).

Going back to our hypothetical, we don't know anything about anything. We could have a giant decision tree to explore with an exponential number of leaves to evaluate.

How does making assumptions help us?

Do we just run a random number generator and if it returns 1, we go "okay, it's satisfiable," and false otherwise?

No.

We make assumptions about our literal assignments.

Unlike brute forcing where we try every possible combination of variable assignments at once, in DPLL we start with a literal, we'll call it A. There is nothing in the equation that indicates whether or not A should be true or false; either could be valid.

So to tackle our problem, initially we will assume A is true and see where that leads us. We propagate that value throughout our clauses, and suppose there is a clause that can't be satisfied. That's a contradiction, because if the equation was satisfiable then $A = \text{True}$ couldn't have led us there.

So we've figured out that at the very least $A = \text{True}$ leads to a contradiction.

We've now eliminated half of the search space immediately and only need to consider the cases where $A = \text{False}$.

This may not make total sense immediately, so instead of giving one explanation of the algorithm and then building it, we will build up the algorithm and explain how each part works.[1]

[2]

3.1 Recursive Backtracking

First, we need to rewrite our brute force algorithm to be recursive, as that's the foundation for the entire algorithm (duh). To do this, we will introduce a new method `solve()` which will be our recursive solving method. To also make things simpler, we will extract the equation evaluation into its own separate method as well.

This means our SAT solver becomes like so:

```
public interface:
    SATSolver(string input)
    int getNumVars()
    int getNumClauses()
    int[][] getClauses()
    optional<bool>[] getSolution()
    void printSolution()
    bool solveCNF()

private interface:
    void parse(string equation)
    bool solve(bool[] assignment)
    bool evaluate(bool[] assignment)
    int numVars
    int numClauses
    int[][] clauses
    optional<bool>[] lits
```

The flow essentially becomes like this: `solveCNF()` calls `solve()`, which calls itself over and over and returns whatever solution it finds back up to `solveCNF()`, who will handle any sort of cleanup.

Why can't we just make `solveCNF()` call itself recursively?

Good question.

[1] Note for future me: write a better introduction.

[2] `https://en.wikipedia.org/wiki/DPLL_algorithm`

That's because `solveCNF()` will handle any setup that `solve()` will need in order to function properly, and it will handle any cleanup that `solve()` may create.

Let's now discuss how `solve()` will work.

It will be passed in a `bool[]` for the current assignment of literals. When the size of our assignment is equal to the number of variables, we will then evaluate the satisfiability of the assignment, and if it's satisfiable we return true. Otherwise, we return false, backtrack, retry, until we either find a satisfiable assignment or we've exhausted them all.

The pseudocode is as so, starting with `evaluate()`:

```
bool evaluate(bool[] assignment)
{
    bool solved = true;

    for each (clause in clauses)
    {
        bool satisfied_clause = false;

        for (j = 1 up to clause.size)
        {
            int variable = clause[j];
            int v_idx = abs(variable);

            if (variable > 0)
            {
                satisfied_clause =
                    assignment[v_idx] || satisfied_clause;
            }
            else
            {
                satisfied_clause =
                    !assignment[v_idx] || satisfied_clause;
            }

            if (satisfied_clause == True)
            {
                continue;
            }
        }

        solved &= satisfied_clause;

        if (solved == False)
        {
            break;
        }
    }

    if (solved == True)
    {
        lits = assignment;
    }
    else
    {
        lits = no-value;
    }
```

```
    return solved;
}
```

As you can see, all we've done is move our equation evaluation from brute force into its own function. One thing to note is that this method also handles updating our `lits` field. Thus, if `evaluate()` returns true, then `lits` will hold the satisfying assignment, and if it returns false, then `lits` will hold nothing.

Now our recursive `solve()` method:

```
bool solve(bool[] assignment)
{
    if (assignment.size == numVars)
    {
        return evaluate(assignment);
    }

    append(assignment, True);

    if (solve(assignment))
    {
        return true;
    }

    assignment.back() = False;

    if (solve(assignment))
    {
        return true;
    }

    truncate(assignment, 1);
    return false;
}
```

This might be slightly confusing at first, so let's go over it.

We first check our base case. If our assignment has all of its variables, we evaluate it and return the result whether it be true or false. And you'll notice that whenever we return true from a call of `solve()`, we immediately return true afterwards, so therefore, if we find a satisfiable assignment, we will go up our call stack back to `solveCNF()`, so no need to worry that it will terminate if we find a satisfiable assignment.

However, if our assignment still needs some variables, we append true to our assignment, then we do a recursive call on `solve()`. If that recursive call leads to a satisfiable assignment, we return true, otherwise, we swap out our last value for false and try `solve()` again. And if that ends up working out, we return true, but if it doesn't then we remove the last value and return false.

So how do we know that it will guarantee termination for an unsatisfiable problem?

Notice how when assigning the current variable true or false and it doesn't work, we just remove the last one and go up the call stack by one? Well suppose we're at the top of the call stack (we've assigned the first variable) and the first variable is assigned false, and that didn't lead to a satisfiable assignment. Then we remove that first variable from the assignment, and then we return false. So our assignment array is empty, and we've returned to `solveCNF()`. Thus, `solve()` will terminate eventually.

And how do we know that we'll try every single assignment?

Well for the very last variable, it's easy to see, if it being true doesn't satisfy the equation, we assign it false, and if that doesn't work, we remove it and go up the call stack. Then for the previous variable, if it was true, we assign it false and go back to the last variable where it tries true and/or false again. Then that doesn't work, we go up the call stack twice, and that continues until we've found a satisfiable assignment or we've exhausted them all.[3]

[3]This is a very informal proof, but what's most important is that we build up our understanding of what's going on instead

Lastly, our `solveCNF()` method:

```
bool solveCNF()
{
    bool[1..numVars] assignment;

    return solve(assignment);
}
```

Because we're dealing with pseudocode, we don't have to do things such as reserving or resizing the assignment array. And since `evaluate()` takes care of making sure our `lits` field holds a valid assignment (if found) and holds nothing if not, we don't need to do anything else in `solveCNF()`.

You may wonder though, why not pass in `lits` to `solve()` or have no setup and make `solveCNF()` purely recursive?

Those are very valid questions to ask.

This has to do with the differences between pseudocode and actual implementation. In the actual implementation, some methods depend on the state of `lits`, such as whether or not it holds anything, and by passing it into `solve()`, we force it to hold something, even if there is no satisfiable assignment. And it's true, that in `solveCNF()` if we had no setup, we could make it recursive on `lits`, however, I chose to implement it differently.

When you are reading this, please ask yourself these kinds of questions, because oftentimes, the way you read my implementation is not the best way, and even if it is, there is no reason as to why you can't think of different implementations, try them out yourself, and compare. Perhaps, and I expect this will happen often, you will find a better implementation than the one written down here.

3.1.1 Recursive Backtracking – Results

Running this code on the same 1000 instances (uf20-91), it took 1,846,644 ms, which is roughly 30.78 minutes. This is actually a pretty big improvement from our 35.32 minutes via brute forcing. Although this seems like a big gain in speed, it should be noted that at this point the algorithm is still just a fancier brute forcing algorithm.

Here's a question for you, why might assigning our variables true instead of false first lead to an increase in speed?

Hint: Look at the number of negations present in the testing set.

3.2 Unit Propagation

Unit propagation is the next step in implementing DPLL, and it introduces the idea of implication.

Imagine clause: $A \to Z$, and we know that $A = \text{False}$. What can we say about Z?

Well, if our equation is satisfiable, then the clause must be satisfiable, therefore $Z = \text{True}$ because if it didn't, that would be a contradiction.

So instead of assigning variables B, C, ..., X, then assigning Z, why don't we just do it now, and use that knowledge in other clauses containing Z or $\neg Z$?

That is essentially the idea behind unit propagation: the decisions we make imply other assignments, and so we should force them now instead of waiting until later.

However, this introduces a light hiccup into our implementation.

You see, our initial invariant about assigning variables is that they are assigned in numerical order instead of (possibly) out of order.

"Not a problem! We'll just make the assignment array have space for every variable so we can index it anywhere," I hear you think.

Unfortunately that interferes with our base case because it relies on the size of the assignment array. If it is initially at a fixed size of `numVars`, then our base case will always fire because our assignment array always

of dumping notation. Perhaps I will do a formal proof in the future.

has a size of `numVars`. So we would either need to change our base case or change how we store learned variables.

What about a learned set? One that holds whether or not we've learned a variable (positive or negative), and then as we traverse down the call stack, we just check if a variable is already learned, and if so, we give it its learned value?

That would 100% work.

In fact, the first time I implemented DPLL, that is exactly what I used.

However, it has the disadvantage of being annoying to maintain.

Instead, why don't we create a new data type called something like `var` that represents the state of each variable: `True`, `False`, and `Unassigned`.

Then we can just maintain a simple counter of how many are not `Unassigned`, and then that keeps our base case nice and neat.

Yes dear reader, we are going to do that, however, if you'd like to use the learned set, reading about how we do it without a learned set will help you and offer clues on how to implement it your way.

3.2.1 Changes To Our Implementation

```
public interface:
    SATSolver(string input)
    int getNumVars()
    int getNumClauses()
    int[][] getClauses()
    optional<bool>[] getSolution()
    optional<var>[] getSolution()
    void printSolution()
    bool solveCNF()

private interface:
    enum var
    {
        False
        True
        Unassigned
    }

    void parse(string equation)
    bool solve(bool[] assignment)
    bool solve(var[] assignment)
    bool evaluate(bool[] assignment)
    bool evaluate(var[] assignment)
    void propagate()

    int numVars
    int numClauses
    int numAssigned

    int[][] clauses

    optional<bool>[] lits
    optional<var>[] vars
```

Here we just add in a new enum for the states that a variable can be, and adjust the methods using `bool`s to use the new enum type instead.

3.2.2 Implementing Unit Propagation

Before we start changing our algorithm, let's implement the `propagate()` method.

As stated before, the idea is that given a clause with either only one variable or only one `Unassigned` variable (and the rest evaluate to `False`), we can immediately say that `Unassigned` variable will be whatever value it needs to be to evaluate to `True` for that clause.

An implementation detail is that typically, once a clause is satisfied because of a literal, you remove every other clause that is satisfied by that literal. And for every other clause containing the negation of the literal, you remove that negation entirely.

This is done to shrink the search space.

Even though I find this to be a waste of memory (copying is expensive), I do see the benefit in reducing the search space, so we will implement it the "textbook" way.[4]

Pseudocode for `propagate()`:

```
void propagate()
{
    Set<int> unit

    int curSize = 0
    int newSize = -1

    int[1..N] unitClauseIdx
    int[1..N] deleteClauseIdx

    while (curSize != newSize)
    {
        curSize = unit.size

        for (i = 1 up to numClauses)
        {
            int numUnassigned = 0
            int unassignedIdx = -1
            bool restFalse = true

            for (j = 1 up to clause.size)
            {
                int var = clause[j]
                int idx = abs(var)

                if (vars[idx] == Unassigned)
                {
                    numUnassigned++
                    unassignedIdx = j
                }

                bool evalsFalse =
                    var < 0 && vars[idx] == True ||
                    var > 0 && vars[idx] == False

                restFalse &= evalsFalse

                if (Contains(unit, -var))
                {
                    RemoveAt(clause, j)
```

[4] https://en.wikipedia.org/wiki/Unit_propagation

```
                }
            }

            if (restFalse == True)
            {
                int var = clause[unassignedIdx]
                int idx = abs(var)

                if (var > 0)
                {
                    assignment[idx] = True
                }
                else
                {
                    assignment[idx] = False
                }
                numAssigned = numAssigned + 1

                Add(unit, var)
                Append(unitClauseIdx, i)
            }
        }

        newSize = unit.size
    }
}
```

3.2.3 Changes To Our Algorithms

The first method we should change is our `solveCNF()` method. Due to the order of our enum labels, the default value for a `var` is `False` rather than `Unassigned`. This allows them to be used as booleans since `False = 0`, `True = 1`, and `Unassigned = 2`. Therefore, we need to manually initialize every variable to be `Unassigned` before we start solving.

Pseudocode for `solveCNF()`:

```
bool solveCNF()
{
    var[1..numVars] assignment;

    for (i = 1 up to numVars)
    {
        assignment[i] = Unassigned;
    }

    numAssigned = 0;

    return solve(assignment);
}
```

Next in `solve()`:

```
bool solve(var[] assignment)
{
    if (numAssigned == numVars)
    {
```

```
        return evaluate(assignment);
    }

    append(assignment, True);
    numAssigned = numAssigned + 1;

    if (solve(assignment) == true)
    {
        return true;
    }

    truncate(assignment, 1);
    numAssigned = numAssigned - 1;

    append(assignment, False);
    numAssigned = numAssigned + 1;

    if (solve(assignment) == true)
    {
        return true;
    }

    truncate(assignment, 1);
    numAssigned = numAssigned - 1;

    return false;
}
```

3.3 Pure Literal Elimination

3.4 Optimizing Our Current Approach

3.5 Iterative DPLL

Chapter 4

CDCL